

Reviewer Pack — Minimal Diophantine Equation Complexity and Communication Ra...

Entry e943d33a6696 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 00:27:13 UTC

Minimal Diophantine Equation Complexity and Communication Rank Variance

Entry ID: e943d33a6696 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 00:27:13 UTC

1. Conjecture

Field A (mathematical branch): Number Theory (Diophantine Equations) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

For every binary boolean function $f: \{0,1\}^n \rightarrow \{0,1\}$, the minimal complexity of a Diophantine equation that computes f is linearly correlated with its communication rank variance ($\text{CRV}(f)$), such that $\min_{g \text{ computes } f} c(g) = \Theta(\text{CRV}(f))$, where $c(g)$ is the complexity of g .

Rationale (proposer's reasoning):

Diophantine equations provide a structured framework for encoding boolean functions, and their complexity may reveal insights into the communication complexity of these functions. The linear correlation suggests a possible link between abstract algebraic structures and concrete computational problems in communication complexity.

Taxonomy category: DiophantineEquations (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d369845a8be8ec0d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson's correlation coefficient between Diophantine equation complexity ($c(g)$) and communication rank variance ($\text{CRV}(f)$) for a sample of 30 random seeds is ≥ 0.8 , with no seed producing $c(g) > 10$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.80	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - title:Diophantine* AND (communication* OR matrix* rank*) - author:(Diophantine* OR communication*) AND (equation* OR rank*) - subject:NUMBER THEORY and (COMMUNICATION COMPLEXITY OR MATRIX RANK) AND Diophantine*

Top relevant hits considered: - [http://arxiv.org/abs/hep-ex/0202036v4] This paper has been withdrawn by the author(s) - [http://arxiv.org/abs/1506.04891v2] Author Identification using Multi-headed Recurrent Neural Networks - [http://arxiv.org/abs/1808.00525v2] The impact of imbalanced training data on machine learning for author name disambiguation - [http://arxiv.org/abs/hep-th/9707234v2] Variational Approach to Quantum Field Theory: Gaussian Approximation and the Perturbative Expansion around It - [http://arxiv.org/abs/chao-dyn/9311011v2] The Great Inequality In A Hamiltonian Planetary Theory - [http://arxiv.org/abs/hep-th/9201009v1] Large-Small Equivalence in String Theory

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
from fractions import Fraction
import math
import itertools

def generate_random_boolean_function(n: int) -> list:
    return [random.choice([0, 1]) for _ in range(2**n)]

def compute_diophantine_equation_complexity(f: list) -> int:
    n = int(math.log2(len(f)))
    if 2**n != len(f):
        raise ValueError("Input list length must be a power of 2")

    matrix = [[Fraction(f[i * (1 << j)], 1) if i & (1 << j) else Fraction(0, 1) for j in range(n)]
    rank = gaussian_elimination(matrix)
    return rank

def gaussian_elimination(matrix: list) -> int:
    rows, cols = len(matrix), len(matrix[0])
    pivot_row = 0
    pivot_col = 0

    while pivot_row < rows and pivot_col < cols:
        # Find the pivot element in the current column
        pivot = None
```

```

    for i in range(pivot_row, rows):
        if matrix[i][pivot_col] != Fraction(0, 1):
            pivot = i
            break

    if pivot is None:
        pivot_col += 1
        continue

    # Swap the pivot row with the current row
    matrix[pivot_row], matrix[pivot] = matrix[pivot], matrix[pivot_row]

    # Eliminate all other elements in the current column
    for i in range(rows):
        if i != pivot_row:
            factor = -matrix[i][pivot_col] / matrix[pivot_row][pivot_col]
            for j in range(cols):
                matrix[i][j] += factor * matrix[pivot_row][j]

    pivot_row += 1
    pivot_col += 1

    return sum(1 for row in matrix if any(x != Fraction(0, 1) for x in row))

def compute_communication_rank_variance(f: list) -> float:
    n = int(math.log2(len(f)))
    if 2**n != len(f):
        raise ValueError("Input list length must be a power of 2")

    # Compute the indicator function for each bit position
    indicators = [sum(1 for x in f if (i >> j) & 1 == x) / len(f) for j in range(n)]

    # Compute the variance of the communication ranks
    mean_rank = sum(indicators)
    variance = sum((rank - mean_rank) ** 2 for rank in indicators) / n
    return variance

def run_trial(seed: int) -> dict:
    random.seed(seed)
    results = []
    for n in [5, 10, 15, 20, 30, 40]:
        f = generate_random_boolean_function(n)
        c_g = compute_diophantine_equation_complexity(f)
        crv_f = compute_communication_rank_variance(f)
        results.append({"n": n, "c_g": c_g, "crv_f": crv_f})

    if not results:
        return {
            "metric_name": "Pearson's correlation coefficient",
            "metric_value": None,
            "instances_tested": 0,
            "n_max": 0,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }

```

```

c_g_values = [result["c_g"] for result in results]
crv_f_values = [result["crv_f"] for result in results]

if any(c_g > 10 for c_g in c_g_values):
    return {
        "metric_name": "Pearson's correlation coefficient",
        "metric_value": None,
        "instances_tested": len(results),
        "n_max": max(result["n"] for result in results),
        "conjecture_holds": False,
        "counterexample": f"c(g) > 10 for n={max(result['n'] for result in results)}"
    }

correlation_coefficient = pearsons_correlation(c_g_values, crv_f_values)
return {
    "metric_name": "Pearson's correlation coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": len(results),
    "n_max": max(result["n"] for result in results),
    "conjecture_holds": correlation_coefficient >= 0.8,
    "counterexample": ""
}

def pearsons_correlation(x: list, y: list) -> float:
    n = len(x)
    if n != len(y):
        raise ValueError("Input lists must have the same length")

    mean_x = sum(x) / n
    mean_y = sum(y) / n

    numerator = sum((x_i - mean_x) * (y_i - mean_y) for x_i, y_i in zip(x, y))
    denominator = math.sqrt(sum((x_i - mean_x) ** 2 for x_i in x)) * math.sqrt(sum((y_i - mean_y) ** 2 for y_i in y))

    if denominator == 0:
        return float('nan')

    return numerator / denominator

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]]
    if not seeds:
        seeds = [2*i - 1 for i in range(5, 6)] # Default to first few primes

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all("conjecture_holds" in r and r["conjecture_holds"] for r in results):
        mean_value = sum(r["metric_value"] for r in results) / len(results)
        std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
        support_fraction = len([r for r in results if "conjecture_holds" in r and r["conjecture_holds"]]) / len(results)
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")

```

```

elif any("counterexample" in r and r["counterexample"] for r in results):
    first_failing_seed = next(r["seed"] for r in results if "counterexample" in r and r["count
    print(f"RESULT: FALSIFIED counterexample=\"{next(r['counterexample'] for r in results if '
else:
    print("RESULT: INCONCLUSIVE")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ffebaf16.py", line 140, in <module>
    result = run_trial(seed)
              ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ffebaf16.py", line 79, in run_trial
    c_g = compute_diophantine_equation_complexity(f)
           ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ffebaf16.py", line 25, in compute_dioph
    matrix = [[Fraction(f[i * (1 << j)], 1) if i & (1 << j) else Fraction(0, 1) for j in range(n)] for i in
               ~~~~~^~~~~~
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means that the Pearson's correlation coefficient could not be calculated. | next: Re-run the test to ensure it completes successfully and produces the required data for calculating the Pearson's correlation coefficient.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15170
2	propose	ollama_remote	glm4:latest	0	0	13083
3	propose	ollama_remote	glm4:latest	0	0	12943
4	preregistration	ollama_remote	glm4:latest	0	0	10769
5	novelty	ollama_remote	glm4:latest	0	0	8553
6	novelty	ollama_remote	glm4:latest	0	0	9660
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18843
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11294

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10831
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17246
11	judge	ollama_remote	glm4:latest	0	0	15403

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 143795 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in research/audit/e943d33a6696.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/e943d33a6696.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/e943d33a6696.tar.gz (if generated)